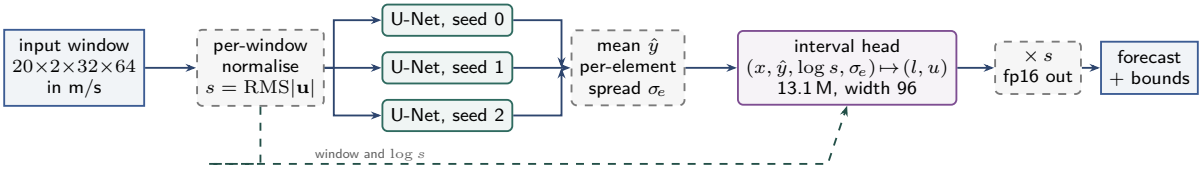
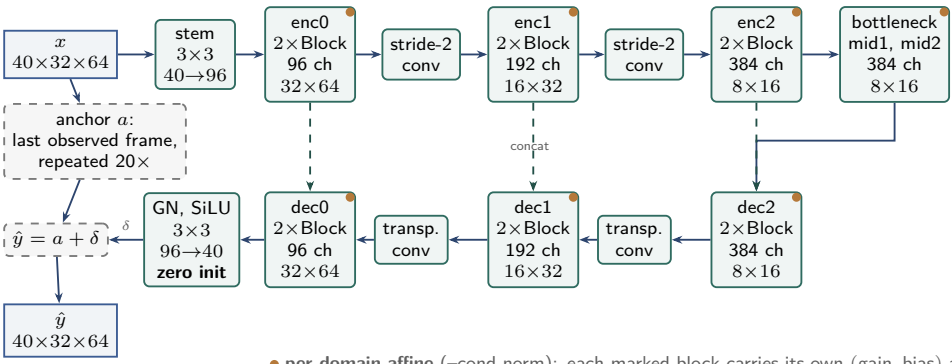
(a) one scored call — 8.02 ms/sample in the evaluation container



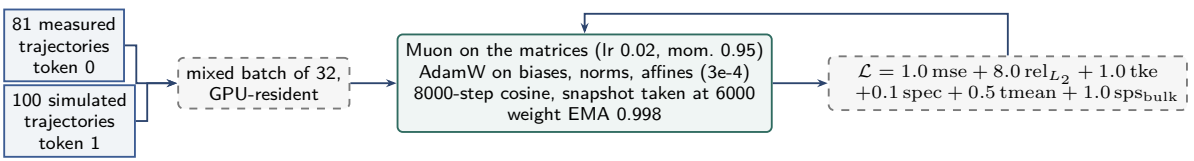
Three members at 22.65 M parameters each plus a 13.1 M head, all in fp16: 162 MB of a 256 MB budget. The scale comes from the input window alone — metadata is an empty dict on scored calls, so neither Reynolds number nor angle of attack is available. It cancels in rel_l2, tke and mvpe, which are per-window ratios; only the SPS bounds have to be mapped back to physical units, because σ_{global} is absolute.

(b) one member — a residual U-Net writing a correction to a persistence anchor



• **per-domain affine** (–cond norm): each marked block carries its own (gain, bias) at both of its GroupNorm sites, selected by a modality token (real = 0, simulated = 1); 28 sites in all. The convolutions stay shared, so simulated windows still teach the real branch its dynamics. Zero-initialised, so training starts at the identity — as does the output convolution, which means the run begins *exactly* at persistence and δ is the whole learned forecast. At export the real row folds into the GroupNorm weights, and the shipped network is a plain U-Net carrying cond: none.

(c) what produces the weights — tools/train_final.sh, ∼1 h on one H100



Every term but spec is the organisers' own scored quantity, taken verbatim from scoring.py. $\mathrm{sps}_{\mathrm{bulk}} = 1 - \overline{\exp(-|e|/\sigma)}$ is the ceiling on the interval reward and the only term that is not an L_2 norm — it weights the bulk of the error distribution where the L_2 terms weight its tail. Simulation enters the *same batch* as measurement rather than a pretraining stage: supplied as an initialisation instead, it is worth 62% as much on held-out angles and exactly nothing at a held-out Reynolds number.